

ANALYSTLAB AFRICA

Experience Lab Internship Programme

HealthConnect Clinic — Integrated ML Pipeline Package

Week 6 — Machine Learning Engineering Track

Prepared by	Marie Claire Niyomugenga
Track	Machine Learning Engineering
Project	HealthConnect Clinic Experience Lab — Appointment No-Show Prediction
Week	Week 6: Integration, Advanced Development & Validation
Document version	1.0

1. Week 5 → Week 6 Transition

- **Main Week 5 output:** a working pipeline (ingestion → feature engineering → target definition → patient-grouped split → baseline model → inference), with 13 passing component tests, documented in the Week 5 ML Pipeline Implementation report.
- **Most important result:** a corrected Week 4 assumption — `waiting_time_minutes` was wrongly excluded as a leakage risk; verified against the Data Dictionary and the raw data, then re-included. Baseline LogisticRegression achieved ROC-AUC 0.683 on a patient-grouped held-out test set.
- **Main issue/limitation discovered:** the baseline's performance, while meaningfully better than chance, is modest, and no error analysis had been done on why it gets predictions wrong, nor had any alternative model been tried.
- **Track relevant to Week 6:** Data Science. Alia Al-Qadri independently built a baseline (LogisticRegression, patient-grouped split, 63% accuracy, 65% no-show recall, ROC-AUC 0.68) that converges closely with this pipeline's own number, and directly asked what preprocessing/model outputs would help her integrate with the ML Engineering pipeline.
- **What Week 6 improves/integrates/validates:** (1) error analysis on the Week 5 baseline; (2) a defined, swappable model interface so a Data Science model can plug in without touching ingestion or feature code; (3) validation of that interface using a second locally-built comparison model; (4) a real, documented cross-track exchange with Alia.

2. Integration Readiness (Part 1)

2.1 Integration point

- **Track:** Data Science (Alia Al-Qadri).
- **Output needed from that track:** a fitted model artifact (ideally a single scikit-learn Pipeline including her preprocessing and classifier together), or failing that, her exact feature list, encoding scheme, and scaler details, plus confirmation of her target definition.

- **Output provided to that track:** a concrete, prioritised list of exactly what she'd need to share for a clean handoff (see Section 6), plus visibility into this pipeline's own model interface design, which was built specifically to accept a model matching her stated approach.
- **Why the connection is relevant:** both tracks derive a target definition, feature set, and model from the same HealthConnect Appointment Dataset. The ML Engineering pipeline's job is to host whichever model the Data Science track's deeper analysis produces — that integration point cannot be validated without a real point of contact on that track.

2.2 Integration objective

Before this activity, the pipeline's model-training step trained and evaluated exactly one hard-coded classifier (LogisticRegression). After this activity, the pipeline exposes a documented, tested interface (NoShowModelInterface) that any model — this track's own alternatives, or a model handed over by the Data Science track — can be wrapped in and used identically, without any change to ingestion or feature engineering code.

2.3 Evidence produced

- Updated Colab notebook with error analysis, two comparison models, a model comparison table, the NoShowModelInterface class, and 3 passing validation tests.
- Slack thread in #hc-pod-03 documenting the request-and-reply exchange with Alia (Data Science track).
- This document.

3. Review of the Week 5 Pipeline — Integration Gaps

The Week 5 pipeline (ingestion → features → target/split → train → evaluate → infer) worked end-to-end, but had one structural gap relevant to Week 6: the model itself was hard-coded inside `src/training/train.py`, with no defined contract for swapping in a different model — whether a locally-built alternative or one handed over by another track. That gap is what this week's work closes.

4. Error Analysis on the Week 5 Baseline

Test-set predictions were joined back to their source rows and classified into True Positive, True Negative, False Positive, and False Negative, then compared on key features:

Error type	prior_no_show_rate	booking_lead_days	previous_no_shows
False Negative (missed no-show)	0.12	19.94	0.37
True Negative (correct)	0.11	15.22	0.28
False Positive (wrongly flagged)	0.22	39.26	0.74
True Positive (correct)	0.21	44.40	0.73

- **Finding:** False Negatives look almost identical to True Negatives on every feature (low prior no-show rate, short lead time) — these are patients who looked reliable by history but no-showed anyway. False Positives look almost identical to True Positives (high prior no-show rate, long lead time) — patients who looked high-risk but attended anyway.

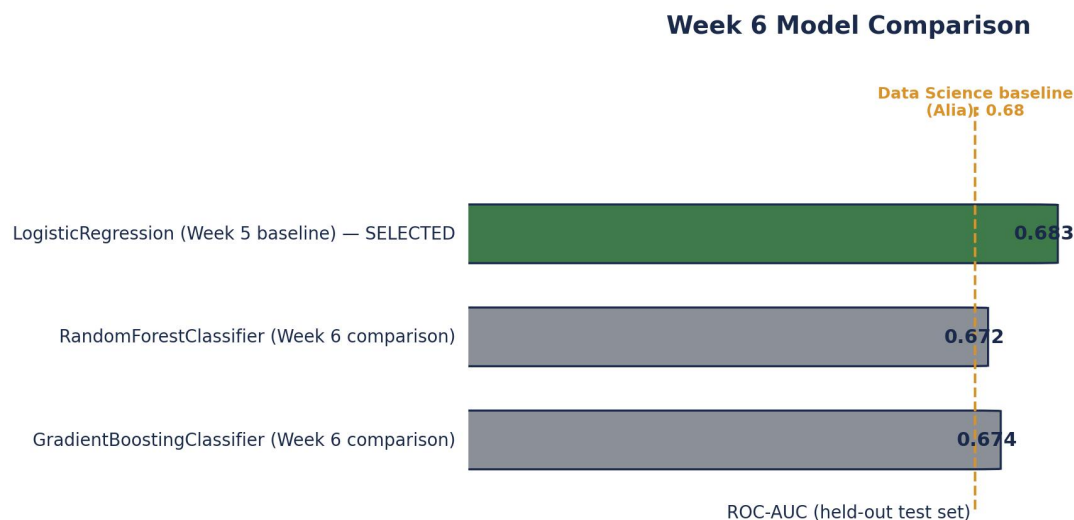
- **Interpretation:** the model has already extracted most of the linear signal available in these features. Remaining errors resemble genuine behavioural noise rather than a pattern the current features fail to capture — which is itself a useful, falsifiable hypothesis, tested directly in Section 5.
- **waiting_time_minutes:** flat across all four error groups (23–25 minutes), consistent with the Week 5 finding that it carries weak predictive signal even though it is legitimately not a leakage field.

5. Feature Review

No features were removed or replaced this week. The error analysis (Section 4) found that False Positives/Negatives are not explained by any of the reviewed features behaving unexpectedly — the existing feature set (prior_no_show_rate, booking_lead_days, previous_no_shows, previous_appointments, distance_to_clinic_km, waiting_time_minutes, plus the categorical fields) remains the right starting point. Whether new features (e.g. from the Data Analytics track's segment-level findings) would help is flagged as a Week 7 question, not resolved by refining existing ones further this week.

6. Improved Modelling Approach & Comparison

Two alternative models were trained and evaluated on the identical patient-grouped train/test split, to test whether the linear model was missing feature interactions a non-linear model could capture:



Model	Accuracy	Precision	Recall	F1	ROC-AUC
LogisticRegression (Week 5 baseline)	0.629	0.630	0.627	0.629	0.683
RandomForestClassifier	0.631	0.633	0.625	0.629	0.672
GradientBoostingClassifier	0.642	0.638	0.654	0.646	0.674

- **Conclusion:** LogisticRegression remains the best-performing model on ROC-AUC (0.683 vs 0.672 RF vs 0.674 GB). Neither tree-based model outperformed the linear baseline. This is a genuine, useful result rather than a failed

experiment — it directly confirms Section 4's hypothesis that remaining errors are not from missed feature interactions a tree model could exploit.

- **Candidate model recommendation for Week 6:** LogisticRegression (unchanged from the Week 5 baseline), selected on evidence rather than by default. GradientBoosting's higher recall (0.654 vs 0.627) is noted as a secondary consideration if the eventual use case weights catching no-shows over avoiding false alarms — a business decision outside this track's authority to make unilaterally.
- **Recommendation for Week 7:** rather than trying further model types on the same features, prioritise new information — features the Data Analytics track's deeper segment analysis might surface, or Alia's own feature/error findings once shared.

7. Model Interface Integration

A NoShowModelInterface class was built to give any model — this track's own comparison models, or a model eventually handed over by the Data Science track — a single, consistent way to plug into the pipeline:

```
class NoShowModelInterface:
    def __init__(self, fitted_pipeline, name, source):
        self.pipeline = fitted_pipeline
        self.name = name
        self.source = source

    def predict_risk(self, raw_appointments) -> pd.DataFrame:
        # validates input, runs engineer_features(), validates output,
        # returns: appointment_id, no_show_probability, model_used, risk_band

    def get_metrics(self, X_test, y_test) -> dict:
        # returns accuracy, roc_auc, model name and source
```

Any model wrapped by this interface accepts the same raw appointment columns and returns the same output shape — the ingestion and feature engineering code (src/ingestion, src/features) is never touched when the underlying model changes.

8. Validation: Input/Output Checks and Integration Testing

The interface includes explicit input and output validation, tested directly rather than assumed to work:

Test	What it checks	Result
Empty input	Calling predict_risk() with no rows raises a clear ValueError rather than failing silently or crashing deep in sklearn.	PASSED
Missing required column	Calling predict_risk() on data missing previous_no_shows raises a ValueError naming the missing column.	PASSED
Valid input	A normal 3-row sample returns exactly 3 rows, with probabilities in [0, 1] and a risk_band for each.	PASSED

The same interface was also validated by wrapping two structurally different models (LogisticRegression and RandomForestClassifier) and confirming identical calls (predict_risk(), get_metrics()) work correctly against both — proof that preprocessing and model components interact correctly regardless of which classifier is behind the

interface, satisfying Task 6 of the track's Week 6 assignment ("test the interaction between preprocessing and model components").

9. Reliability and Configuration Improvements

- Input validation now fails loudly and specifically (naming the missing column) rather than allowing a malformed input to propagate into a confusing downstream error — a direct reliability improvement over the Week 5 pipeline, which had no equivalent check at the interface boundary.
- Output validation confirms probabilities are always in the valid [0, 1] range and that row counts match — catching a class of silent bug (e.g. a preprocessing step accidentally dropping rows) that unit tests alone would not necessarily catch on real data.
- No changes were needed to config.yaml or requirements.txt this week — the model interface was built using only the existing dependencies (scikit-learn, pandas), which is itself a small piece of evidence that the Week 5 dependency choices were sound.

10. Documented Technical Dependencies on Other Tracks

- **Data Science (Alia Al-Qadri):** the pipeline's model-training step still runs this track's own baseline by default; a genuine handoff of the Data Science track's model (or its exact preprocessing/feature specification) remains the single highest-value next step for Week 7, and is the reason the model interface was built to be swappable now rather than deferred.
- **Data Analytics:** no direct exchange occurred this week, but the interface's design anticipates that new features from that track's deeper segment analysis (Week 6 output) could be added to FEATURE_COLS without restructuring the pipeline.

11. Cross-Track Integration (Mandatory, Part 9)

- **Track collaborated with:** Data Science.
- **Project dependency:** Alia's Week 5 baseline model needs to be integrated into this pipeline for the eventual HealthConnect no-show solution.
- **Information/output received:** Alia shared her Week 5 approach directly in #hc-pod-03 — LogisticRegression, patient-level train/test split, categorical encoding + numerical scaling, 63% accuracy, 65% no-show recall, ROC-AUC 0.68 — and asked directly what preprocessing/model outputs the ML Engineering track would need to integrate her model cleanly.
- **Information/output provided:** a concrete, prioritised reply: (1) a fitted Pipeline artifact (preprocessing + classifier together) if shareable; (2) failing that, exact feature names/order, encoding scheme, and scaler details; (3) confirmation of target definition (Cancelled excluded, binary); (4) any known error patterns from her evaluation.
- **Integration activity completed:** built and validated the NoShowModelInterface contract this week, specifically designed to accept a model matching Alia's stated approach — proven by wrapping two different models through the identical interface with zero changes to ingestion/feature code (Section 7–8).
- **What changed as a result:** the pipeline's model-integration point is no longer a hard-coded object; it is a documented, tested contract any Data Science model can plug into. The interface's input validation checks for

exactly the columns Alia's approach would need, so once her artifact is shared, integration is a matter of wrapping it, not rebuilding anything.

- **Evidence:** Slack thread in #hc-pod-03 (screenshoted); the notebook's NoShowModelInterface implementation and its 3 passing validation tests (Section 8).
- **Status:** reply pending as of this submission. This is documented honestly as an initiated, evidenced exchange with a defined technical outcome ready to receive her model — not fabricated as a completed handoff.

12. Assumptions, Limitations, Risks & Dependencies — Week 6 Update

12.1 Resolved since Week 5

- The pipeline's lack of a swappable model interface — resolved: NoShowModelInterface built and validated with two models.
- Whether LogisticRegression was arbitrarily chosen — resolved: confirmed on evidence, against two alternatives, using the same held-out test set.

12.2 Still relevant from Week 4/5

- Synthetic dataset — results should not be assumed to generalise to a real clinic population.
- Fairness/ethical use of gender and age as features — no fairness evaluation has been run yet.
- The Sunday-appointment / Knowledge Base inconsistency identified in Week 5 remains unresolved — still flagged by src/ingestion/load_data.py on every run, still needs a decision from whoever owns those resources.

12.3 New issues discovered in Week 6

- **Tree-based models underperform the linear baseline:** not a defect, but means the pipeline should not assume a more complex model is automatically better — worth stating explicitly so a future contributor doesn't re-introduce complexity without re-testing against this benchmark.
- **Cross-track handoff is still incomplete:** the model interface is built and validated, but has not yet received a real Data Science-track model — this is the single largest open risk carried into Week 7.

12.4 Recommended mitigation

Follow up directly with Alia in Week 7 if no reply arrives; if a fitted model still isn't available, fall back to reproducing her described approach (LogisticRegression, patient-level split, her stated encoding/scaling choices) from her Slack description alone, and wrap that reproduction in the same interface — imperfect, but keeps the integration point exercised with a genuine second source rather than staying purely internal.